



Today's agenda → DP on strings

- ↳ longest common subsequence
- ↳ problems based on above
- ↳ Edit distance.



AlgoPrep



Q) **LCS** → longest common subsequence

↳ Given 2 strings, find the length of longest common subsequence.

Ex1: S1: a b b c d g b
S2: b a c h e g b

Seq: acgb → 4

Ex2: S1: k r a g o i p
S2: l g i g k m
Seq: lgi → 3

Idea 1

↳ Get all the subsequences of S1 as well S2, find the common with max length.

T.C: $O(2^N)$



1/idea2

s_1 : a b b c ~~d~~ g b
 s_2 : a c h e g b

$LCS(s_1[0-6], s_2[0-5])$

if ($s_1[6] == s_2[5]$)

$1 + LCS(s_1[0-5], s_2[0-4])$

if ($s_1[5] == s_2[4]$)

$1 + LCS(s_1[0-4], s_2[0-3])$

if ($s_1[4] != s_2[3]$)

$\max(x, y)$

$LCS(s_1[0-3], s_2[0-3])$

$LCS(s_1[0-4], s_2[0-2])$

$LCS(s_1[0-2], s_2[0-3])$

$LCS(s_1[0-3], s_2[0-2])$

$LCS(s_1[0-3], s_2[0-2])$



// Pseudo code $\rightarrow$ {Top-down approach}

```
int dp[N][M] = {-1};  
int LCS (String s1, int i, String s2, int j) {  
    if (i == -1 || j == -1) {  
        return 0;  
    }  
    if (dp[i][j] != -1) { return dp[i][j]; }  
    if (s1[i] == s2[j]) {  
        int x = LCS(s1, i-1, s2, j-1);  
        dp[i][j] = x+1;  
        return x+1;  
    }  
    else {  
        int x = LCS(s1, i-1, s2, j);  
        int y = LCS(s1, i, s2, j-1);  
        dp[i][j] = max(x, y);  
        return max(x, y);  
    }  
}
```

T.C: $O(N \times M)$

S.C: $O(N \times M)$



Iterative soln (bottom-up approach)

S1: a b b c d g b
S2: a c h e g b

		a	c	h	e	g	b
		0	1	2	3	4	5
a	0	1	1	1	1	1	1
b	1	1	1	1	1	1	1
b	2	1	1	1	1	1	1
c	3	1	2	2	2	2	2
d	4	1	2	2	2	2	2
g	5	1	2	2	2	3	3
b	6	1	2	2	2	3	4

$$dp[i][j] = \begin{cases} \text{if } (s1[i] == s2[j]) \rightarrow dp[i-1][j-1] + 1; \\ \text{if } (s1[i] \neq s2[j]) \rightarrow \max(dp[i-1][j], dp[i][j-1]); \end{cases}$$



Q) Longest Palindromic Subsequence

↳ Given a String, find the length of longest Palindromic Subsequence.

Ex: A L G O P R E P A

↓
A P E P A → length = 5

S1: P E P] → Palindrome String :: rev(Palindrome String)
S2: P E P]

→ S1: A L G O P R E P A]
S2: A P E R P O G L A] find longest Common Subsequence.

Longest Palindromic Subsequence = LCS (String, reverse of String);



Q) LCS of 3 strings.

Ex: $S_1: a b c d f e g$
 $S_2: b c f g e a$
 $S_3: b d f e a$

Diagram showing the LCS of S_1 and S_2 is $b c f e$. Then, comparing $b c f e$ with S_3 , the common characters are b, f, e , so the final LCS is $b f e$ with length 3.

Idea 1 → incorrect idea

↳ find LCS of first 2 strings → S_4

↓
 find LCS of S_4 and S_3 → length = ans

$S_1: a e b$
 $S_2: b e f a$
 $S_3: a g h y z$

Diagram showing the LCS of S_1 and S_2 is $e b$. Then, comparing $e b$ with S_3 , there are no common characters, so the final LCS is empty with length 0.

max $[LCS(LCS(S_1, S_2), S_3), LCS(LCS(S_1, S_3), S_2),$
 $LCS(LCS(S_2, S_3), S_1)]$



Idea 2

↳ we need to process all 3 strings together.

s_1 : a b c d f e g ^{i}
 s_2 : b c f g e a ^{j}
 s_3 : b d f e a ^{k}

$$\textcircled{i} \quad s_1[i] == s_2[j] == s_3[k]$$

$$\text{LCS}(s_1[0, i], s_2[0, j], s_3[0, k]) = 1 + [\text{LCS}(s_1[0, i-1], s_2[0, j-1], s_3[0, k-1])]$$

max(x, y, z)

$$\textcircled{ii} \quad s_1[i] != s_2[j] \parallel s_2[j] != s_3[k] \parallel s_1[i] != s_3[k]$$

$$\text{LCS}(s_1[0, i-1], s_2[0, j], s_3[0, k])$$

$$\text{LCS}(s_1[0, i], s_2[0, j-1], s_3[0, k])$$

$$\text{LCS}(s_1[0, i], s_2[0, j], s_3[0, k-1])$$



//Pseudo code

```
int dp[N][M][K] = {-1};
```

```
int LCS (String s1, int i, String s2, int j, String s3, int k) {  
    if (i == -1 || j == -1 || k == -1) {  
        return 0;  
    }  
}
```

```
if (dp[i][j][k] != -1) { return dp[i][j][k]; }
```

```
if (s1[i] == s2[j] && s2[j] == s3[k]) {
```

```
    int n = LCS(s1, i-1, s2, j-1, s3, k-1);
```

```
    dp[i][j][k] = n+1;  
    return n+1;  
}
```

```
else {
```

```
    int n = LCS(s1, i-1, s2, j, s3, k);
```

```
    int y = LCS(s1, i, s2, j-1, s3, k);
```

```
    int z = LCS(s1, i, s2, j, s3, k-1);
```

```
    dp[i][j][k] = max(n, y, z);
```

```
    return max(n, y, z);  
}
```

```
}
```

T.C: $O(N \times M \times K)$

S.C: $O(N \times M \times K)$



Edit distance $\rightarrow$ {Google}

Given 2 strings S_1 and S_2 , min operation to be performed in S_1 so that it becomes S_2 .

Operation:

- (i) we can insert any char at any index in S_1 .
- (ii) we can replace any char with any other char.
- (iii) we can delete any char at any index in S_1 .

Ex: $S_1: \text{d b a c l}$ $\rightarrow$ ans=3

$S_2: \text{b g l}$

$S_1:$ d b a c l
 $S_2:$ b g l

$ED(S_1[0, 4], S_2[0, 2])$

$S_1[4] == S_2[2]$

$ED(S_1[0, 3], S_2[0, 1]) \rightarrow \min(x+1, y+1, z+1);$

$S_1[3] != S_2[1]$

add

$z+1$

replace

$y+1$

delete

$x+1$

$ED(S_1[0, 3], S_2[0, 0])$

$ED(S_1[0, 2], S_2[0, 0])$

$ED(S_1[0, 2], S_2[0, 1])$



// Pseudo code

int dp[N][m] = {-1};

```
int Editdistance (String s1, int i, String s2, int j) {
    if (i == -1 && j == -1) { return 0; }
```

```
    if (i == -1 && j != -1) { return j+1; }
```

```
    if (i != -1 && j == -1) { return i+1; }
    if (dp[i][j] != -1) { return dp[i][j]; }
```

T.C: $O(N \times m)$

S.C: $O(N \times m)$

```
    if (s1[i] == s2[j]) {
        int x = Editdistance (s1, i-1, s2, j-1);
        dp[i][j] = x;
        return x;
    }
```

```
    else {
```

```
        int x = Editdistance (s1, i-1, s2, j); //delete
```

```
        int y = Editdistance (s1, i-1, s2, j-1); //replace
```

```
        int z = Editdistance (s1, i, s2, j-1); //add
```

```
        dp[i][j] = min (x, y, z);
```

```
        return min (x, y, z) + 1;
```

```
    }
```

```
}
```